From the Computer Vision Club, BITS Goa

Hand Tracking with MediaPipe

Week 4 — Beginner Reference Guide

Theory and code snippets for filling in the three TODOs

★ How to use this document.

This short guide covers exactly what you need to complete the Week 4 assignment — nothing more. Read it once before opening the boilerplate, then jump back to any section when you're stuck on a specific TODO. Every concept is explained simply, with a ready-to-use code example.

Contents

1 What Is MediaPipe?

2 The 21 Hand Landmarks

3 TODO 1 — Create the HandLandmarker

- 3.1 BaseOptions
- 3.2 HandLandmarkerOptions
- 3.3 Building the object

4 TODO 2 — Detect Hands in Each Frame

- 4.1 Convert BGR to RGB
- 4.2 Wrap the frame as a MediaPipe Image
- 4.3 Call detect_for_video

5 TODO 3 — Draw the Landmarks

6 The Webcam Loop (already written for you)

7 Quick Reference Cheat-Sheet

Chapter 1 What Is MediaPipe?

MediaPipe is a free library from Google that can detect faces, bodies, and hands in images and video using pre-trained AI models. For this assignment you will use its **Hand Landmarker** — a ready-made detector that finds 21 key points on each hand in your webcam feed.

Install both libraries with one command:

```
pip install opencv-python mediapipe
```

Import them at the top of your script:

```
import cv2
import mediapipe as mp
from mediapipe.tasks import python as mp_python
from mediapipe.tasks.python import vision as mp_vision
```

★ Note

The model file (`hand_landmarker.task`) is downloaded automatically the first time you run the script — you don't need to do anything extra.

Chapter 2 The 21 Hand Landmarks

MediaPipe places **21 numbered dots** on each detected hand — one at the wrist, then four along each finger (base → middle → top → tip). The `HAND_CONNECTIONS` list in the boilerplate tells the script which pairs of dots to connect with a line.

Definition 2.1 — Landmark coordinates

Each landmark has an **x** and a **y** value. These are *not* pixel numbers — they are fractions between 0 and 1 (0 = left/top edge, 1 = right/bottom edge). To get the actual pixel position you multiply by the frame size:

```
h, w, _ = frame.shape
px = int(landmark.x * w) # pixel column
py = int(landmark.y * h) # pixel row
```

■ Watch Out

Never pass the raw x/y values to OpenCV functions — they are fractions, not pixels. Always multiply by the width/height and convert to `int` first.

Chapter 3 TODO 1 — Create the HandLandmarker

Before the webcam loop starts you need to build the detector object. This is a three-line pattern — think of it as: *where is the model → how should it run → build it*.

3.1 BaseOptions — where is the model?

Definition 3.1 — mp_python.BaseOptions

Tell MediaPipe where to find the downloaded model file:

```
base_options = mp_python.BaseOptions(model_asset_path=MODEL_PATH)
```

3.2 HandLandmarkerOptions — how should it run?

Definition 3.2 — mp_vision.HandLandmarkerOptions

Configure the detector. The two settings you need are **running_mode** (use VIDEO for a webcam loop) and **num_hands** (how many hands to track at once):

```
options = mp_vision.HandLandmarkerOptions(
base_options=base_options,
running_mode=mp_vision.RunningMode.VIDEO,
num_hands=2,
)
```

★ Note

Use `RunningMode.VIDEO` whenever you are feeding a continuous stream of frames (like a webcam). It lets MediaPipe track hands across frames for better accuracy.

3.3 Build the object

Definition 3.3 — HandLandmarker.create_from_options

Pass the options to the factory method to get a ready-to-use detector:

```
landmarker = mp_vision.HandLandmarker.create_from_options(options)
```

■ Watch Out

Create the landmarker *once*, before the loop — not inside it. Building it loads the AI model from disk, which is slow. Calling it inside the loop would make your program almost freeze.

✓ In The Assignment

TODO 1 is exactly these three lines. Uncomment the hint block in the boilerplate and you're done. Also uncomment `landmarker.close()` at the very bottom.

Chapter 4 TODO 2 — Detect Hands in Each Frame

Inside the loop, for every webcam frame, you need to do three things: convert the colour, wrap the frame, and run the detector.

4.1 Convert BGR to RGB

Definition 4.1 — cv2.cvtColor — colour conversion

OpenCV stores frames in BGR (Blue-Green-Red) order, but MediaPipe expects RGB (Red-Green-Blue). One line fixes this:

```
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

★ Note

You saw this same conversion in Week 3 when using Matplotlib. Keep using the original `frame` (BGR) for drawing — only pass `rgb_frame` to MediaPipe.

4.2 Wrap the frame as a MediaPipe Image

Definition 4.2 — mp.Image

MediaPipe can't accept a plain NumPy array — wrap it in an `mp.Image` object first:

```
mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)
```

4.3 Call detect_for_video

Definition 4.3 — detect_for_video

Run the detector. In VIDEO mode the function also needs a **timestamp** (how many milliseconds since the program started) so MediaPipe can track motion across frames:

```
timestamp_ms = int((time.time() - start_time) * 1000)
result = landmarker.detect_for_video(mp_image, timestamp_ms)
```

`result.hand_landmarks` is a list of detected hands. Each item in the list is itself a list of 21 landmark objects. If no hand is visible the list is empty.

✓ In The Assignment

TODO 2 is these three steps in order: cvtColor → mp.Image → detect_for_video. Uncomment the hint block — the lines slot straight in.

Chapter 5 TODO 3 — Draw the Landmarks

The last TODO draws the skeleton on the frame whenever a hand is detected. The `draw_landmarks()` helper is already written for you — you just need to call it in the right place.

Definition 5.1 — Iterating over detected hands

`result.hand_landmarks` is a list — one entry per hand found. Loop over it and pass each hand's landmarks to the helper:

```
if result.hand_landmarks:
for hand_landmarks in result.hand_landmarks:
draw_landmarks(frame, hand_landmarks)
```

★ Note

The `if` check is just good practice — it skips the loop when no hand is in frame. Without it the code still works fine because looping over an empty list does nothing.

What draw_landmarks does (already written for you):

```
def draw_landmarks(frame, hand_landmarks):
h, w, _ = frame.shape
# Convert each landmark to pixel coordinates
points = [(int(lm.x * w), int(lm.y * h)) for lm in hand_landmarks]

# Draw lines between connected landmarks (green)
for start_idx, end_idx in HAND_CONNECTIONS:
cv2.line(frame, points[start_idx], points[end_idx], (0,255,0), 2)

# Draw a dot at each landmark (red, filled)
for x, y in points:
cv2.circle(frame, (x, y), 4, (0,0,255), -1)
```

✓ In The Assignment

TODO 3 is the three-line block above (the if + for + draw_landmarks call). Uncomment the hint block and you're done — the helper does all the actual drawing.

Chapter 6 The Webcam Loop (already written for you)

You don't need to change the webcam loop — but understanding it helps you debug problems.

Definition 6.1 — Opening the webcam

`cv2.VideoCapture(0)` opens your first connected camera. Passing 0 means 'first webcam'. `.read()` grabs one frame at a time:

```
cap = cv2.VideoCapture(0)
success, frame = cap.read()
```

Definition 6.2 — Mirroring the frame

`cv2.flip(frame, 1)` flips the image horizontally so it acts like a mirror — your right hand appears on the right side of the screen. This happens *before* detection so the landmarks match what you see.

```
frame = cv2.flip(frame, 1)
```

Definition 6.3 — Quitting with 'q'

`cv2.waitKey(1)` checks for a keypress every frame. When you press 'q' the loop breaks and everything is closed cleanly.

```
if cv2.waitKey(1) & 0xFF == ord('q'):
break
```

■ Watch Out

If the webcam window doesn't open, another app (Zoom, Teams, Camera) is probably already using it. Close those apps and try again.

Chapter 7 Quick Reference Cheat-Sheet

What	Code
Install libraries	`pip install opencv-python mediapipe`
Point to model	`mp_python.BaseOptions(model_asset_path=PATH)`
Set options (VIDEO mode)	`mp_vision.HandLandmarkerOptions( base_options=base_options, running_mode=RunningMode.VIDEO, num_hands=2)`
Build detector	`landmarker = HandLandmarker.create_from_options(options)`
Open webcam	`cap = cv2.VideoCapture(0)`
Read frame	`success, frame = cap.read()`
Mirror frame	`frame = cv2.flip(frame, 1)`
BGR → RGB	`rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`
Wrap as MediaPipe Image	`mp_img = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)`
Get timestamp (ms)	`ts = int((time.time() - start_time) * 1000)`
Run detection	`result = landmarker.detect_for_video(mp_img, ts)`
Loop over hands	`for hand_lms in result.hand_landmarks:`
Normalised → pixel	`px, py = int(lm.x * w), int(lm.y * h)`
Draw line	`cv2.line(frame, pt1, pt2, (0,255,0), 2)`
Draw dot	`cv2.circle(frame, (x,y), 4, (0,0,255), -1)`
Show frame	`cv2.imshow('title', frame)`
Quit on 'q'	`if cv2.waitKey(1) & 0xFF == ord('q'): break`
Clean up	`cap.release() cv2.destroyAllWindows() landmarker.close()`

#GO ERC!!!!!